





23 JAN 2025

DLthon “Jellyfish Classification”

길 찾는 오리들

| 강소안, 김태훈, 윤형석, 이종현, 오병철





Table of contents

01 프로젝트 개요

02 데이터셋 소개 및 분석

03 데이터 라벨링 검증 및 전처리

04 모델 선정 및 비교

05 모델 학습 및 Transfer Learning

06 결론 및 향후 개선 사항

07 Q&A



01

Project overview

Jellyfish Classification

주제: 해파리 이미지 분류

데이터셋 분석

데이터 전처리 및
증강 기법

모델 선정 및
비교

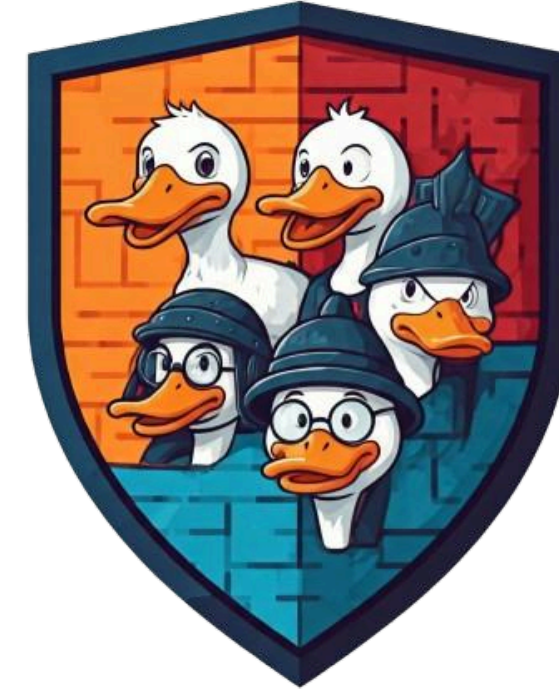
W&B
하이퍼파라미터
튜닝

모델 평가 지표
및 성능 분석

최적 모델
선정



Team Introduction



강소안

AI Model
Baseline Model
DL Model Tuning



김태훈

AI Model
W&B
Flutter



윤형석

AI Model
Transfer Learning
FastAPI



이종현

AI Model
Data Preprocessing
W&B



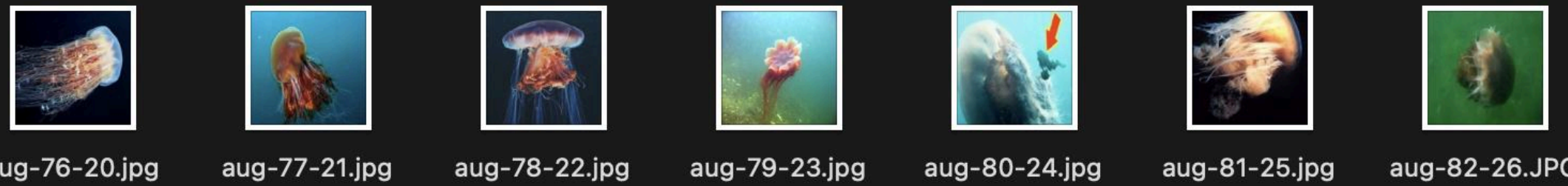
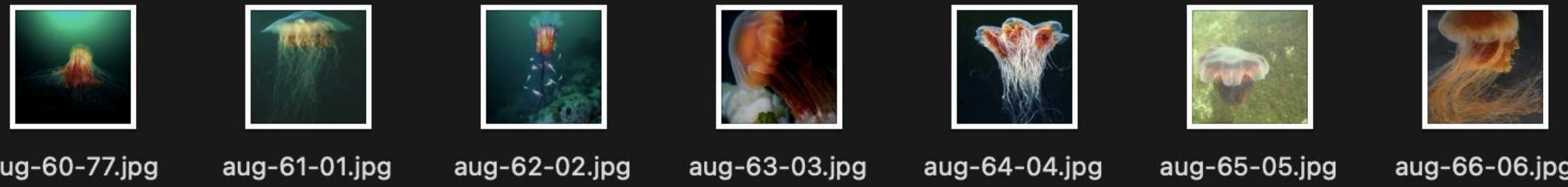
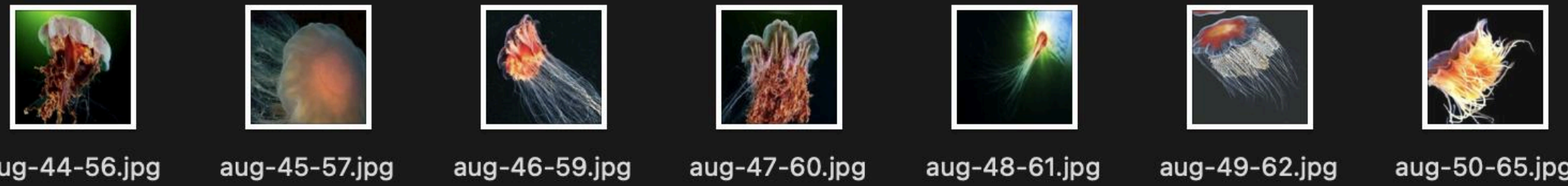
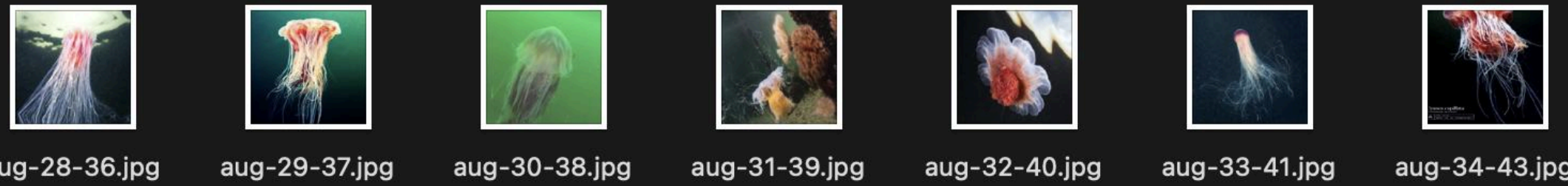
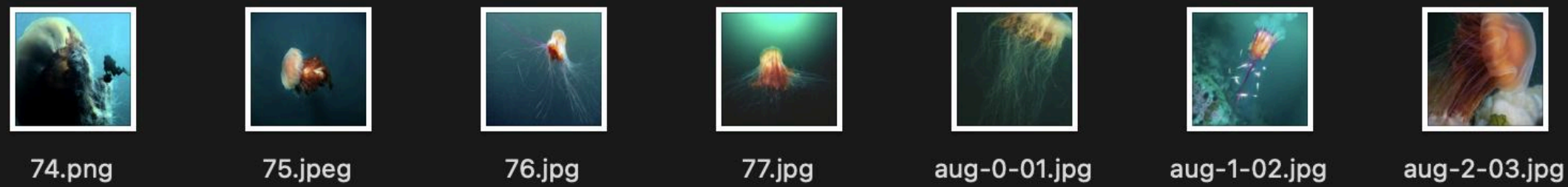
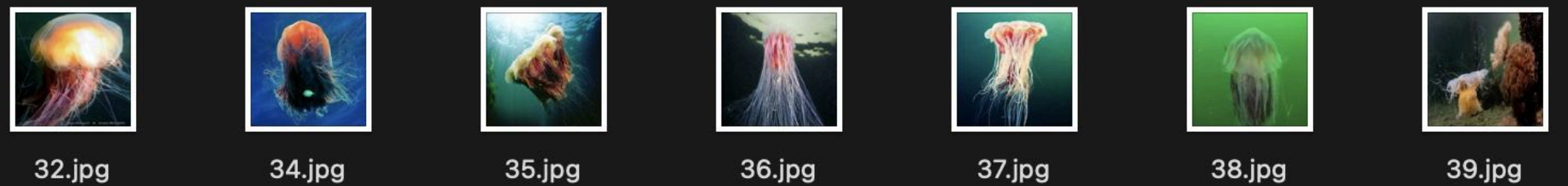
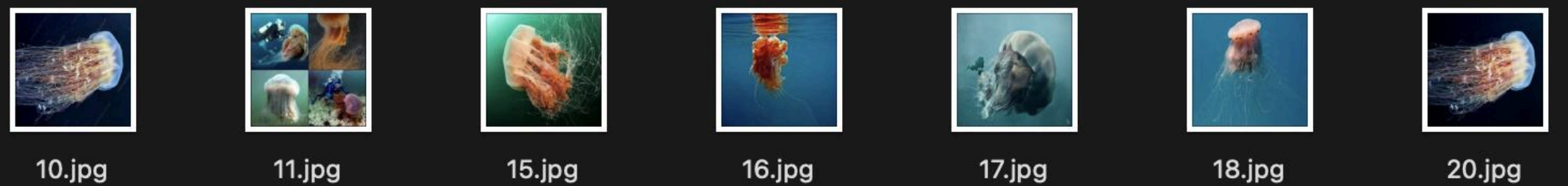
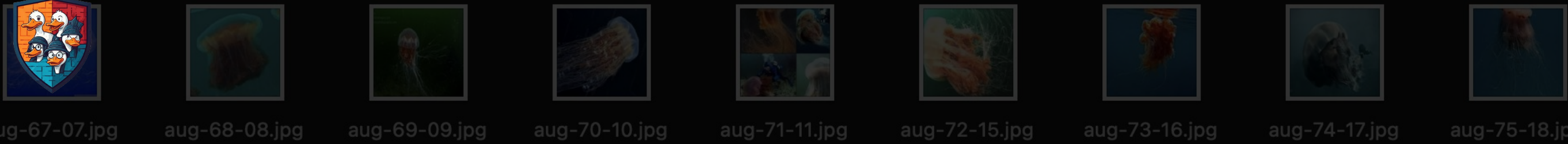
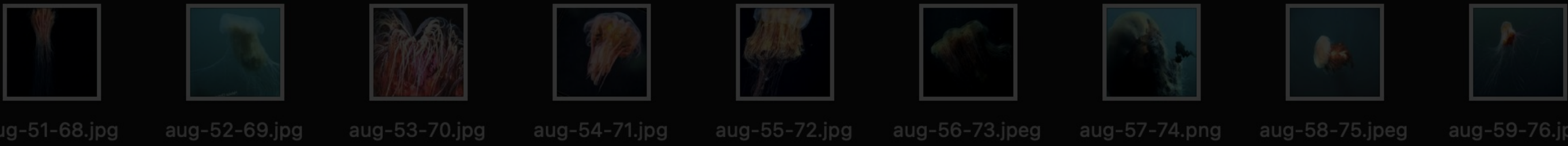
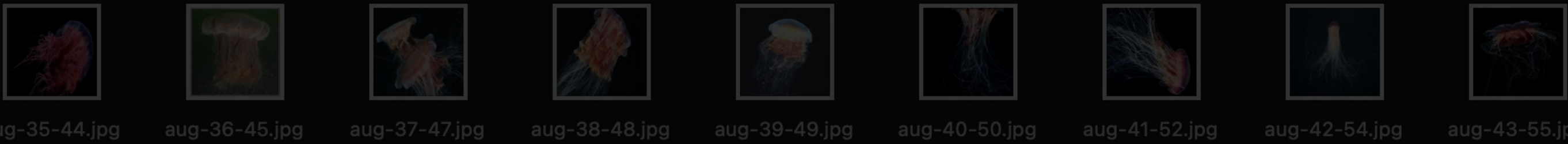
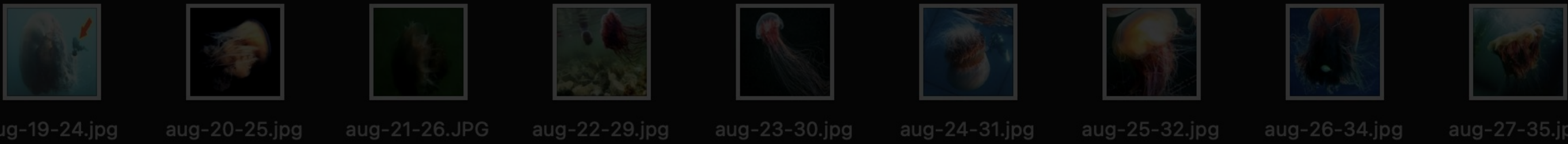
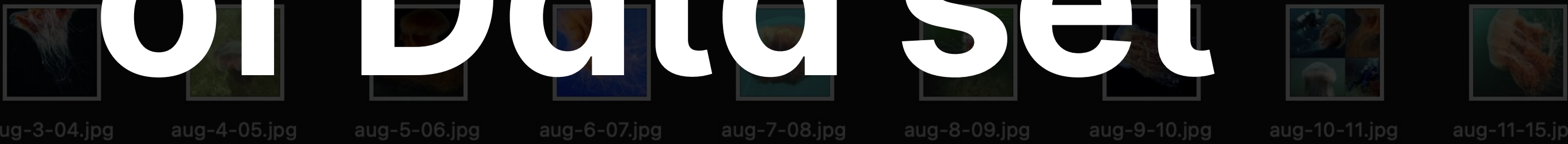
오병철

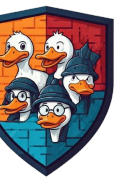
Project Management
Flutter
Presentation



02

Introduction & Analysis of Data set





02

데이터셋 다운로드

우선, 캐글에서 제공하는 해파리 데이터셋을 다운로드했다.

```
kaggle datasets download -d anshtanwar/jellyfish-types
```

+ :: 다운로드한 파일을 압축 해제하면 7개의 폴더가 생성된다.

1. 6개의 폴더는 해파리의 각 종을 나타내며, 폴더 이름은 해파리 종의 이름으로 되어 있다.
2. Train_Test_Valid 폴더는 훈련, 검증, 테스트 데이터가 이미 분류된 형태로 구성되어 있다. 내부 구조는 다음과 같다:
 - Train 폴더
 - test 폴더
 - valid 폴더
 - 각 폴더 내부에는 다시 해파리 종별로 나뉜 클래스 폴더가 존재하며, 해당 클래스의 이미지가 저장되어 있다.

< > blue_jellyfish

이름
02.jpg
03.jpg
04.jpg
06.jpg
07.jpg
08.jpg
09.jpg

< > test

이름
> barrel_jellyfish
> blue_jellyfish
> compass_jellyfish
> lions_mane_jellyfish
> mauve_stinger_jellyfish
> Moon_jellyfish

< > Train_Test_Valid

이름
> test
> Train
> valid

aug-0-01.jpg

aug-1-02.jpg

EXPLORATIONS

데이터셋 구성 및 특징

이미지 분류 작업을 위해, 데이터셋에 대한 충분한 이해와 분석은 필수적입니다.

데이터셋 다운로드 및 구성

- 데이터 출처: 캐글
- 데이터 구조:
 - 6개의 클래스 폴더
 - Train_Test_Valid 폴더:

데이터 특징

- 증강 데이터:
 - Train 폴더에만 포함, 파일명은 aug- 접두사를 가짐.
 - 예: aug-o-01.jpg는 01.jpg를 증강하여 생성된 데이터.
- 총 데이터 개수:
 - 클래스 폴더: 약 900개 이미지.
 - Train_Test_Valid 폴더: 1000개 이상.



02

데이터 분석 및 가설 검증

주요 가설

- "6개의 클래스 폴더는 전체 데이터셋을 포함한다."
- "클래스 폴더는 훈련 데이터만 포함하며, Train_Test_Valid 폴더는 검증 및 테스트 데이터를 포함한다."

검증 결과

- Train 데이터와 클래스 데이터의 일치 확인:
 - 클래스 폴더와 Train 폴더 데이터는 완전히 동일.
- 검증 및 테스트 데이터 중복 확인:
 - Train 데이터와 test, valid 데이터는 중복 없음.
 - test, valid 데이터는 독립적인 데이터셋.

증강 데이터와 원본 데이터의 비율 확인

- 캐글 데이터 셋은 원본 대 증강의 비율이 대략 1 : 2 정도로 형성되어 있음
- 각 클래스가 150개로 일정하게 훈련 데이터가 들어있어서, 적절한 증강 비율과 클래스별 균형잡힌 데이터를 구성하는 이유에 대해 알아보고 고민함.

클래스명	총 데이터	원본 데이터	증강 데이터	증강/원본 비율	비율 평가
barrel_jellyfish	150	41	109	2.66	적절
blue_jellyfish	150	55	95	1.73	적절
compass_jellyfish	150	57	93	1.63	적절
lions_mane_jellyfish	150	61	89	1.46	적절
mauve_stinger_jellyfish	150	58	92	1.59	적절
Moon_jellyfish	150	47	103	2.19	적절



03

데이터 라벨링 검증 및 전처리





03

데이터 라벨링 검증

- **데이터 검증 중요성**

- 훈련 데이터의 라벨링 정확성은 모델 성능에 결정적 영향을 미칩니다.
- 클라우드소싱 기반 데이터는 라벨링 오류 가능성이 있으므로 철저한 검증이 필요합니다.

- **검증 및 개선 접근법**

- 샘플링 기반 검토
- 사전 학습 모델 4가지 활용
- 육안 검사 (워터마크, 바닥에 뭉개진 해파리, 해파리 말고 다른 객체, 이미지 왜곡, 다수 해파리, 클래스 별 이미지 전체 특징 패턴에 맞지 않는 이미지)

- **느낀 점**

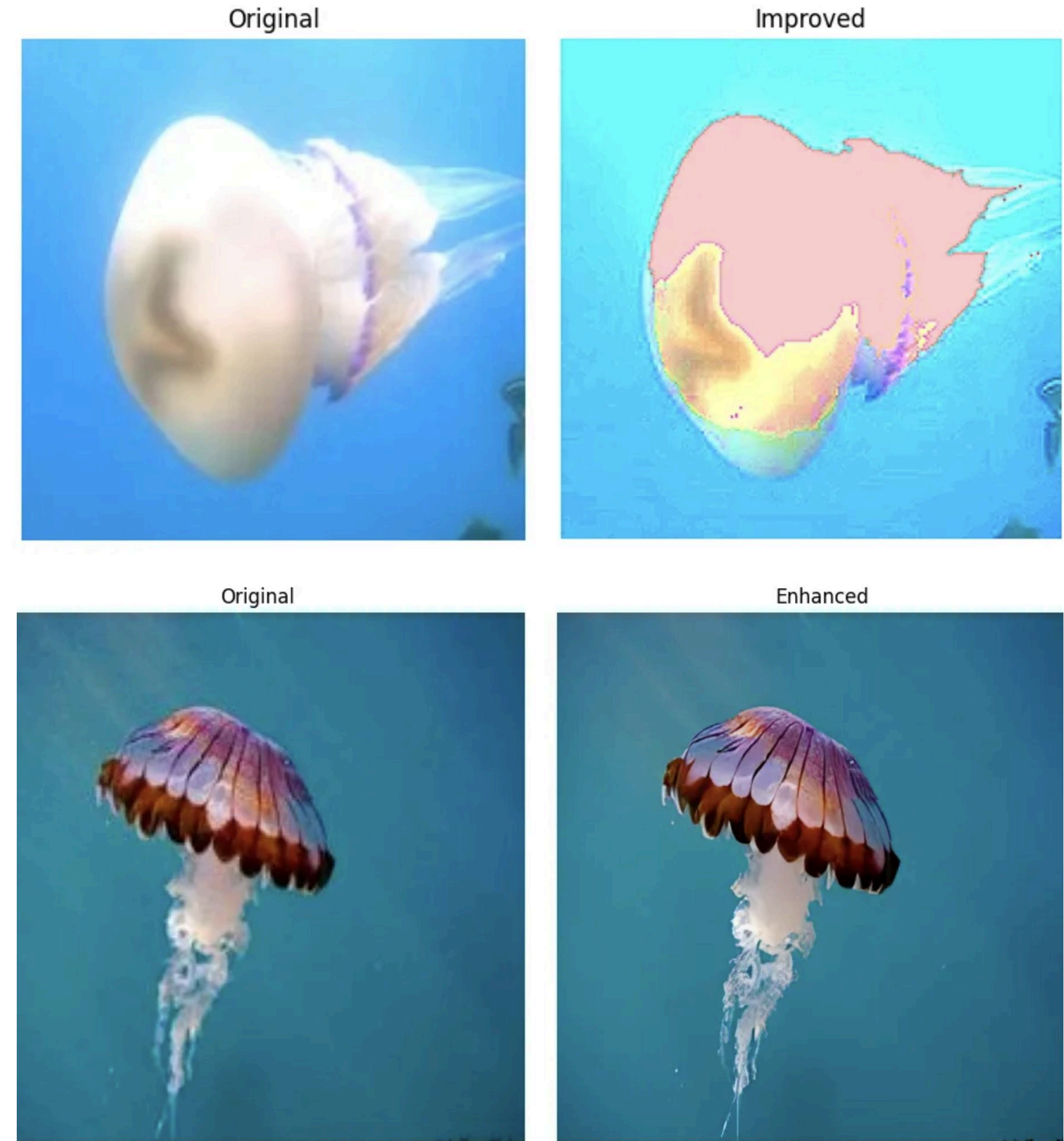
- 모델링에 맞는 양질의 데이터를 수집할 수 있는 인프라의 중요성
- 그럼에도 양질의 데이터가 아닌 대부분의 데이터를 이용해 유용한 인사이트를 도출해 낼 수 있게 전처리를 해낼 수 있는 인프라 구성도 중요함

03

이미지 품질 확인 및 개선

데이터의 중요성을 실감한 후 추가적인 전처리로 이미지의 해상도를 올려서 성능 개선에 도움을 줄 수 있을지 하나의 가설을 세우고 새로운 품질 개선 데이터 셋을 구성 시도함.

1. 밝기, 채도, 대비, 노이즈, 샤프니스 값 개선
 - a. 해파리 부분을 세그멘테이션 해서 해당 부분에서 평균적으로 값이 부족하다고 보여지는 부분을 개선해서 이미지의 성능을 올려보려고 시도함.
 - b. 결과적으로 굉장히 부자연스러운 이미지가 생성됨
2. Real-ESRGAN 라이브러리 활용
 - a. 충분히 검증된 라이브러리라고 판단되어서 이미지의 용량을 올리면서 훈련 시간이 오래 걸리더라도 해상도를 올림으로써 성능 개선이 된다면 충분히 의미있는 과정이라고 생각되어서 시도함.
 - b. 간단한 모델 테스트로는 유의미한 성능 개선이 전혀 보이지 않음. 오히려 약간 떨어지는 경향도 보임.





03

데이터 라벨링, 검증 및 품질 개선을 위한 자동화 파이프라인 구성

1. 이미지 데이터 라벨링: 품질 평가 및 교정

- 데이터 특성 기반 검증
 - 알고리즘 활용: 이미지 데이터의 시각적 특징(예: 색상, 텍스처, 패턴)을 분석하여, 라벨링된 데이터가 이러한 특성과 일치하는지 자동 검증.
 - 예시: 동일한 클래스에 속한 이미지들의 특징 벡터를 클러스터링하여, 비일관성을 감지.
- AI 기반 품질 교정
 - 사전 학습된 모델 활용: ResNet, EfficientNet 등 강력한 이미지 분류 모델로 부정확한 라벨링 탐지.
 - 다중 라벨 평가: 여러 작업자 또는 모델이 생성한 라벨을 통계적으로 분석하여 최적의 라벨 선택.
 - 활용 사례: Amazon SageMaker Ground Truth는 작업자 간 라벨 불일치를 자동으로 조정.
- 반복적 학습 (Iterative Learning)
 - 피드백 루프: 모델이 예측한 라벨과 기존 라벨의 차이를 기반으로, 품질 개선을 위한 재라벨링 요청.

출처

-  SUPERB AI 데이터 라벨링 자동화의 주요 과제와 이를 극복하는 방법
-  mymaster 이미지 데이터 처리 및 전처리 기법: 딥러닝 모델 학습 위한 필수 과정 - 데이터분석 전문가 칼럼
-  FasterCapital 데이터 라벨링 품질: 데이터 라벨링 품질 보증: 스타트업의 비밀병기 - FasterCapital
-  능동 학습이란? – 정의, 장점, 작동 방식, 인사이트 | appen 에펜



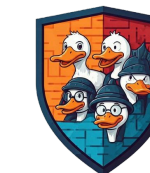
데이터 라벨링, 검증 및 품질 개선을 위한 자동화 파이프라인 구성

2. 다크 데이터 탐지 및 정제

- 다크 데이터 탐지
 - 이상치 감지: DBSCAN 같은 군집화 알고리즘으로 이미지 데이터의 비정상적 샘플 탐지.
 - 자동 태깅: 데이터의 메타정보(예: 이미지 생성 시간, 카메라 메타데이터)를 분석하여 다크 데이터 식별.
- 정제 자동화
 - 노이즈 제거: 이미지의 화질 개선을 위한 Gaussian Blur, Median Filter 등의 기술 적용.
 - 데이터 증강: 라벨된 데이터 부족 문제 해결을 위해 이미지 회전, 크롭, 밝기 조정 등 변환 적용.

3. 데이터 품질 개선 사례

- Google's TensorFlow Extended (TFX)
 - 데이터 전처리, 모델 학습, 배포를 통합 관리하며, 이미지 데이터 품질 평가 및 변환에 적합한 파이프라인 제공.
- Hugging Face Datasets
 - 이미지 데이터셋의 필터링, 변환, 분석을 지원하며, 특성 기반 라벨링 품질 점검 가능.
- AWS SageMaker Ground Truth
 - 반자동 라벨링 및 작업자 결과 비교로 라벨 품질 관리.



모델 선정 및 비교





Baseline Model 학습 및 기본 성능 확인

1단계

- 교재 코드 8-1 활용
- 기본 CNN 모델을 적용하여 초기 성능 측정

2단계

- 모델 고도화, 추가적인 구조 개선 실험
- 정확도를 62.49%까지 향상

3단계

- 모델 조합을 통한 **성능 개선 모듈화** 적용
- Transfer Learning을 통해 **추가적인 성능 개선 가능성 확인**

추진 내용		성능평가		Summary		
		Accuracy	Loss	Total	Trainable	Optimizer
(1단계) 가장 단순한 모델 구축						
	기본 컨브넷 (코드8-1)	18.92 %	8.2945	6,509,780	2,169,926	4,339,854
(2단계) 모델 고도화						
	+ 컨볼루션 층 추가 (코드8-7)	36.49 %	5.8252	3,396,308	1,132,102	2,264,206
	+ 리스케일링 추가	45.95 %	4.9315			
	구조개선 : 배치 정규화 및 패딩 적용	50.00 %	2.2790	258,068	86,022	172,046
	+ 드롭아웃 추가(0.5) -	52.70 %	1.5471			
	+ k-Fold 적용(5)	58.09 %	1.7738			
	+ 데이터 증강 (에포크:30)	60.93 %	1.0971			
	+ 에포크 확대 (100)	62.49 %	1.6945			
(3단계) CNN 기반 모델 비교						
	ResNet50	70.27 %	2.2495	49,279,622	25,691,910	-
	EfficientNetB0	75.68 %	1.0104	20,107,689	16,058,118	-



04

기본 컨브넷 (코드8-1) ↓

(코드8-7) →

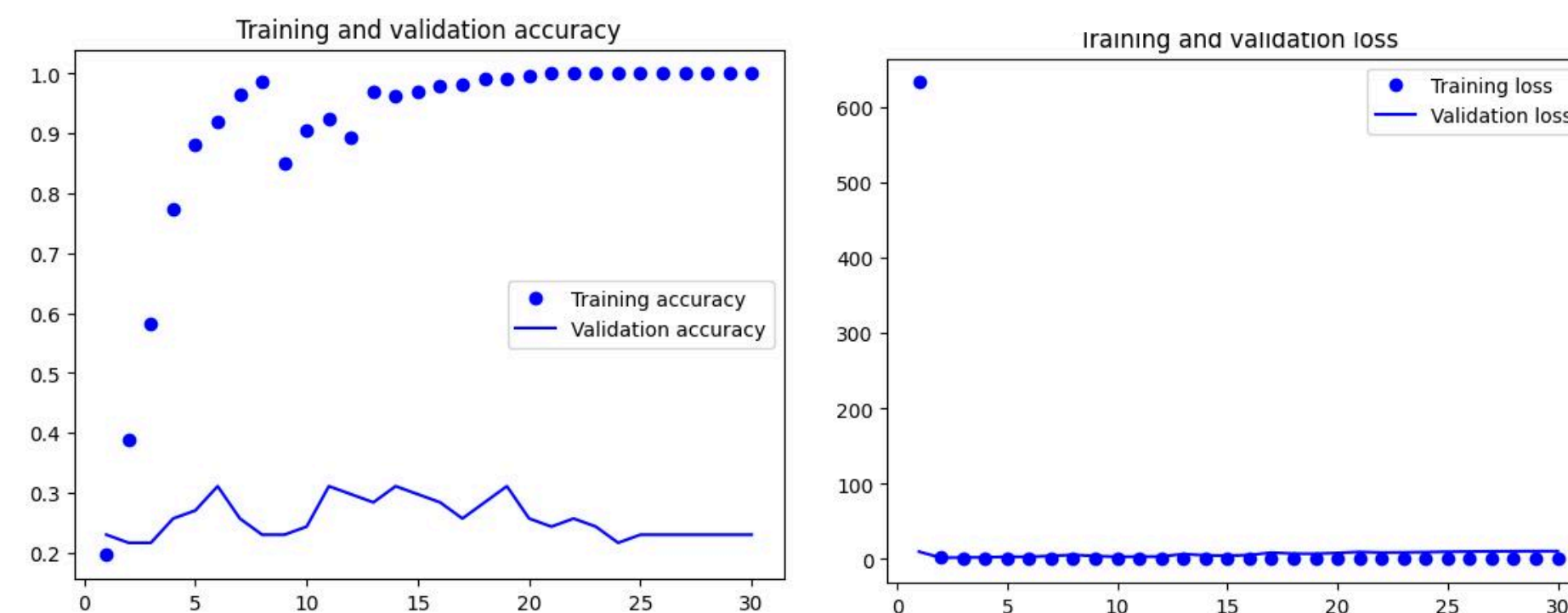
+ Rescaling

```
inputs = tf.keras.Input(shape=(224, 224, 3))
x = tf.keras.layers.Conv2D(filters=32, kernel_size=3, activation="relu")(inputs)
x = tf.keras.layers.MaxPooling2D(pool_size=2)(x)
x = tf.keras.layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = tf.keras.layers.MaxPooling2D(pool_size=2)(x)
x = tf.keras.layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = tf.keras.layers.Flatten()(x)
outputs = tf.keras.layers.Dense(len(datasets['train_ds'].class_names), activation='softmax')
model = tf.keras.Model(inputs=inputs, outputs=outputs)
```

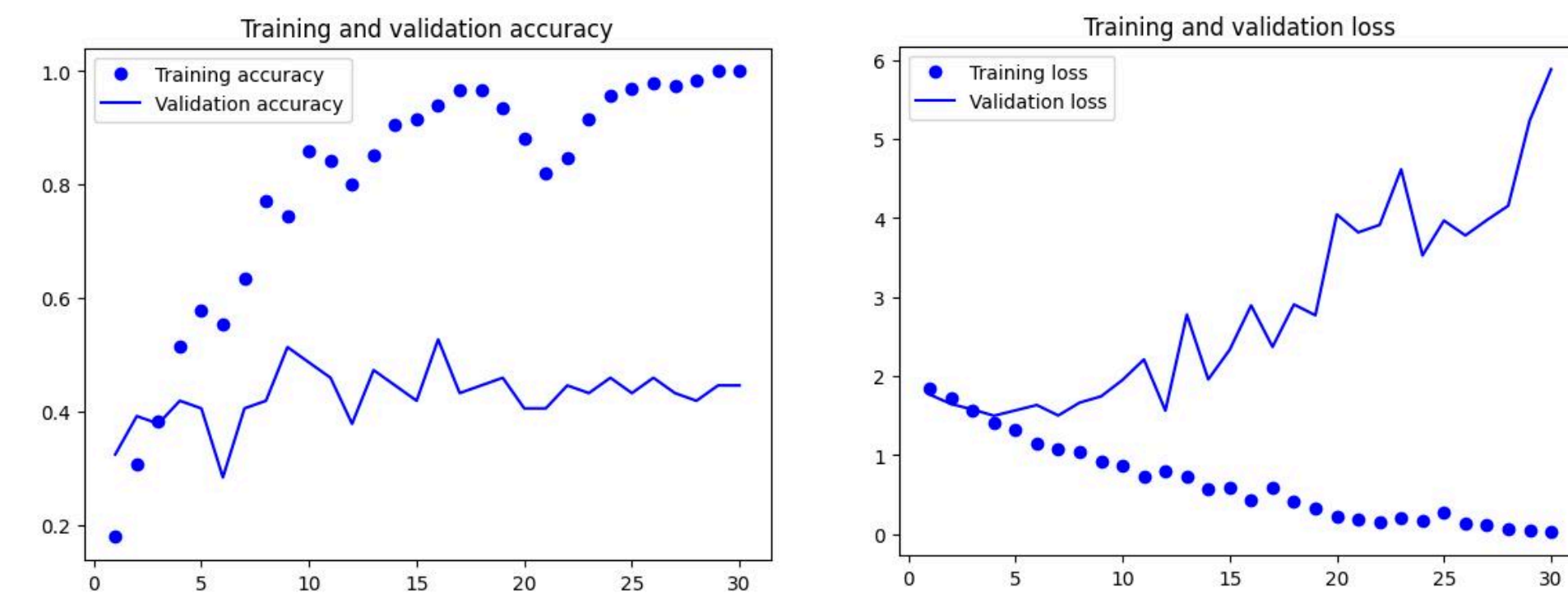
```
inputs = tf.keras.Input(shape=(224, 224, 3))
x = tf.keras.layers.Rescaling(1./255)(inputs)
x = tf.keras.layers.Conv2D(filters=32, kernel_size=3, activation="relu")(x)
x = tf.keras.layers.MaxPooling2D(pool_size=2)(x)
x = tf.keras.layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = tf.keras.layers.MaxPooling2D(pool_size=2)(x)
x = tf.keras.layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = tf.keras.layers.MaxPooling2D(pool_size=2)(x)
x = tf.keras.layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = tf.keras.layers.MaxPooling2D(pool_size=2)(x)
x = tf.keras.layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = tf.keras.layers.Flatten()(x)
outputs = tf.keras.layers.Dense(len(datasets['train_ds'].class_names), activation='softmax')
model = tf.keras.Model(inputs=inputs, outputs=outputs)
```

Test 평가

• Accuracy: 18.92 % Loss : 8.2945



• Accuracy: 45.95 % Loss : 4.9315





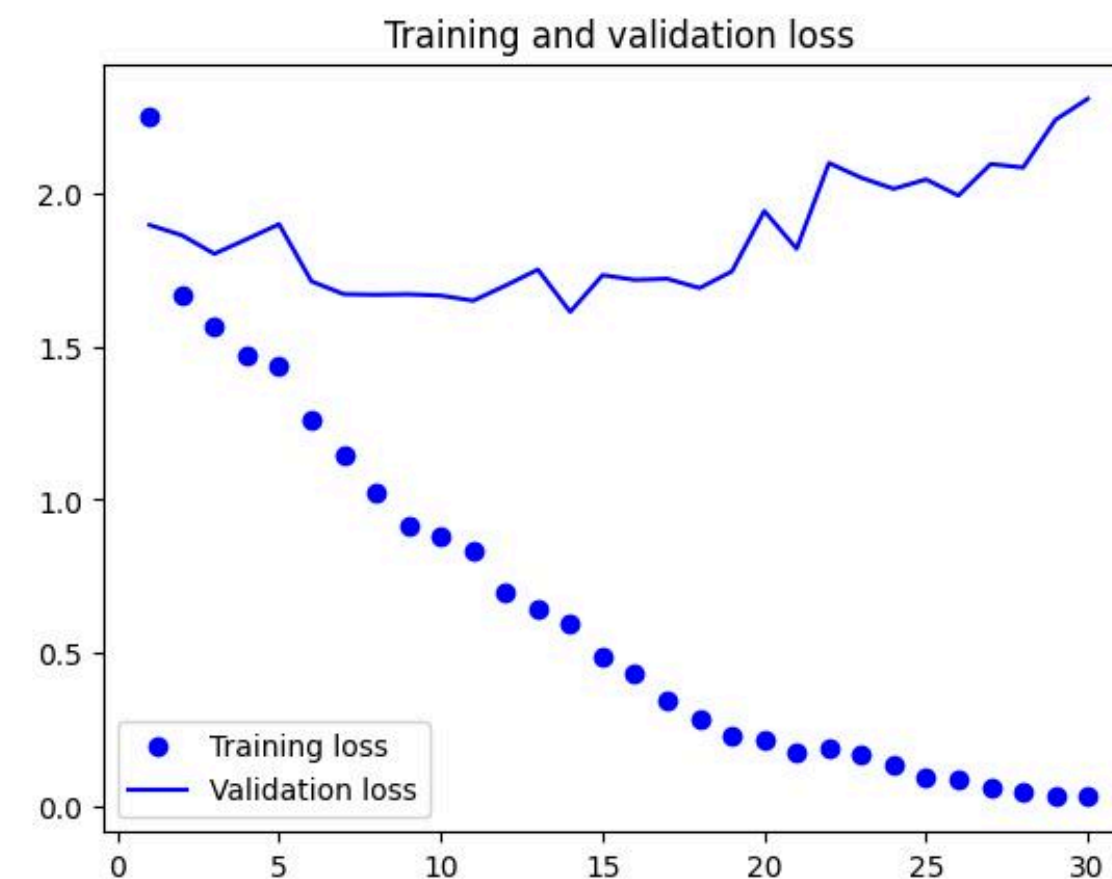
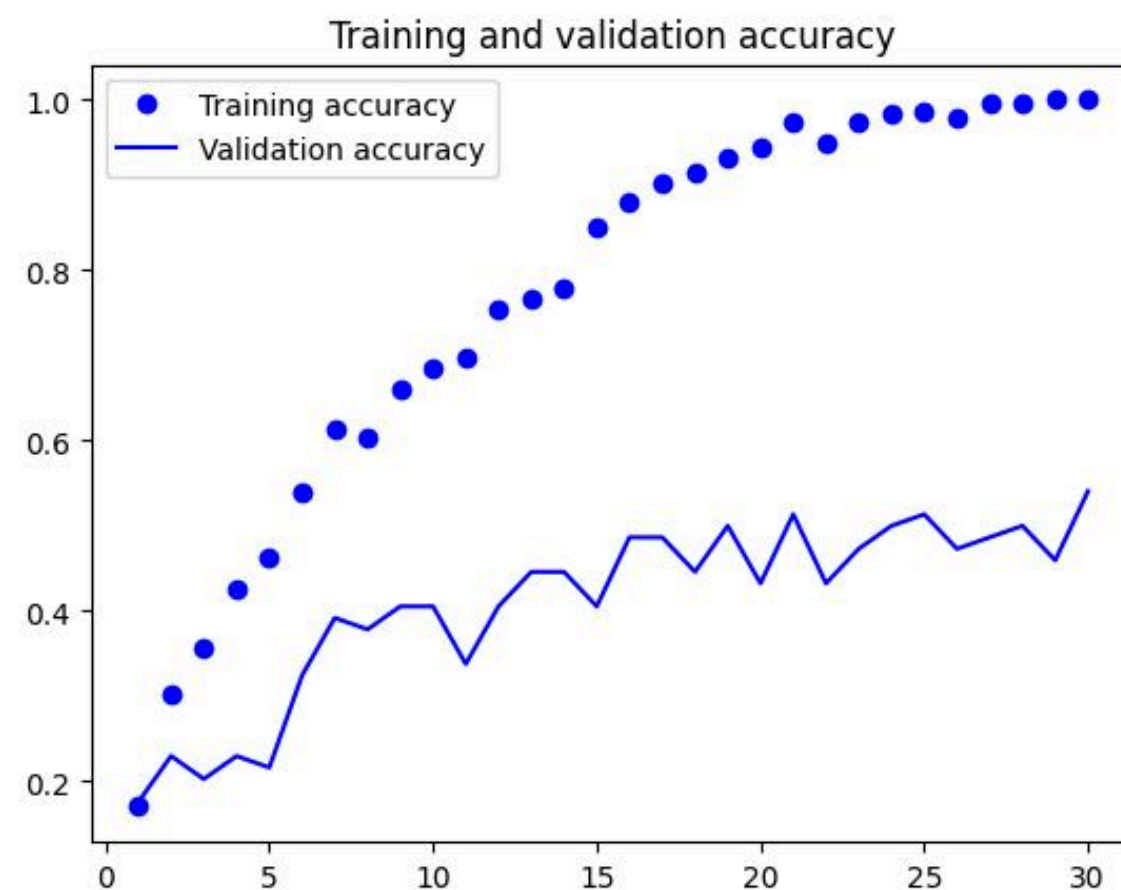
04

배치 정규화 / 패딩 적용

+ Dropout

• Accuracy: 52.70 % Loss : 1.5471

```
inputs = tf.keras.Input(shape=(224, 224, 3))
x = tf.keras.layers.Conv2D(32, (3,3), activation='relu', padding='same')(inputs)
x = tf.keras.layers.LayerNormalization()(x)
x = tf.keras.layers.MaxPooling2D((4,4), padding='same')(x)
x = tf.keras.layers.Conv2D(32, (3,3), activation='relu', padding='same')(x)
x = tf.keras.layers.LayerNormalization()(x)
x = tf.keras.layers.MaxPooling2D((4,4), padding='same')(x)
x = tf.keras.layers.Conv2D(32, (3,3), activation='relu', padding='same')(x)
x = tf.keras.layers.LayerNormalization()(x)
x = tf.keras.layers.MaxPooling2D((4,4), padding='same')(x)
x = tf.keras.layers.Flatten()(x)
x = tf.keras.layers.Dense(128, activation='relu')(x)
x = tf.keras.layers.Dropout(0.5)(x), # 추가
outputs = tf.keras.layers.Dense(len(datasets['train_ds'].class_names), activation='so
model = tf.keras.Model(inputs=inputs, outputs=outputs)
```



데이터 증강 + K-Fold

• Accuracy: 60.93 % Loss : 1.0971

```
Python ▾
# 데이터 증강
def image_preprocessing_v1():
    return keras.Sequential([
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(0.2),
        layers.RandomTranslation(0.1, 0.1),
        layers.RandomZoom(0.2),

        layers.Rescaling(1./255),
    ])
```

+ epochs 상향 조정 (30 → 100)

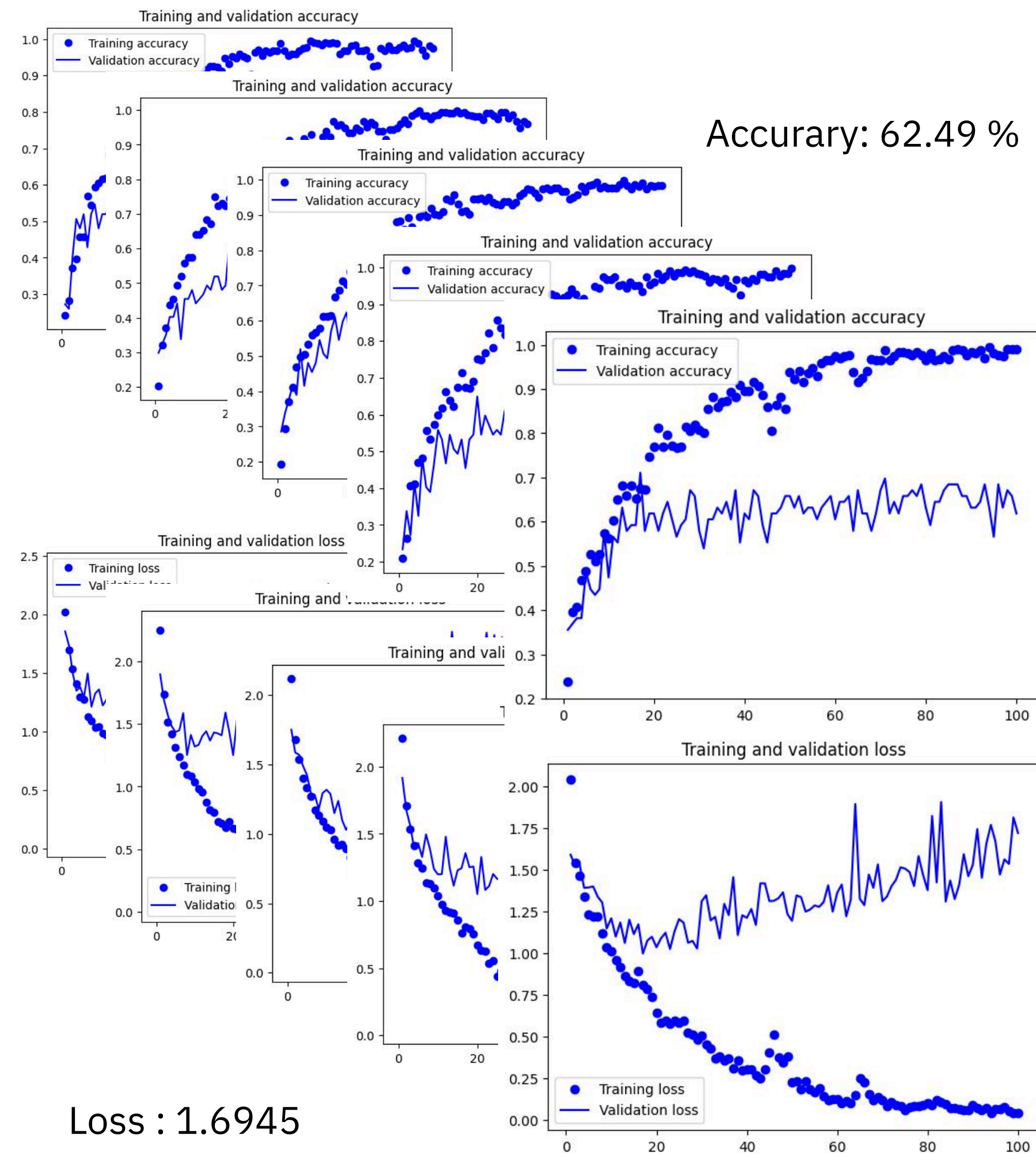
• Accuracy: **62.49 %** Loss : 1.6945

```
Python ▾
# 모델 학습
model_history = model.fit(
    train_generator,
    epochs=100,
    validation_data=validation_generator,
    callbacks=[checkpoint_callback]
)
```



04

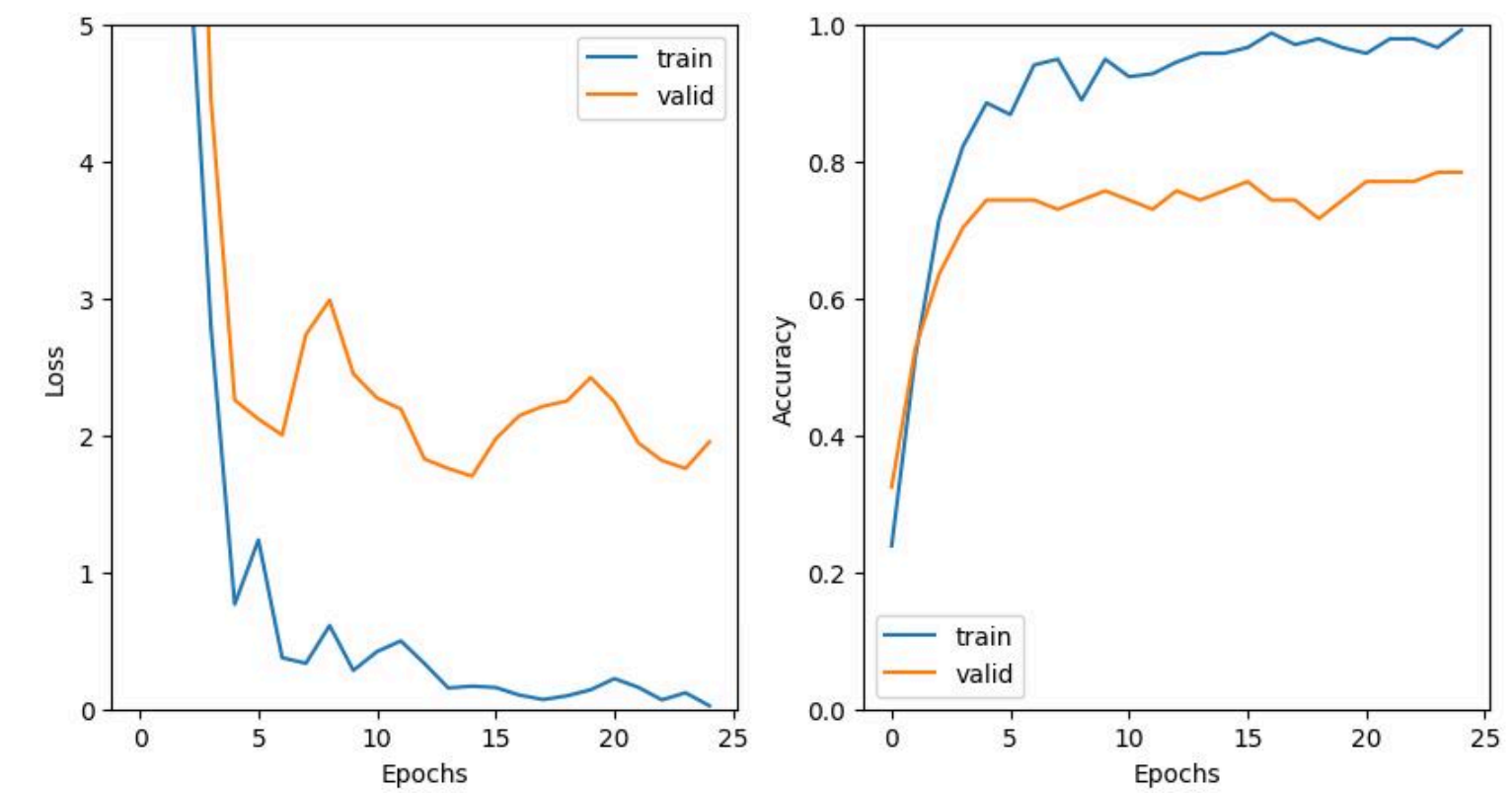
K-Fold + 데이터 증강



CNN기반 모델 비교

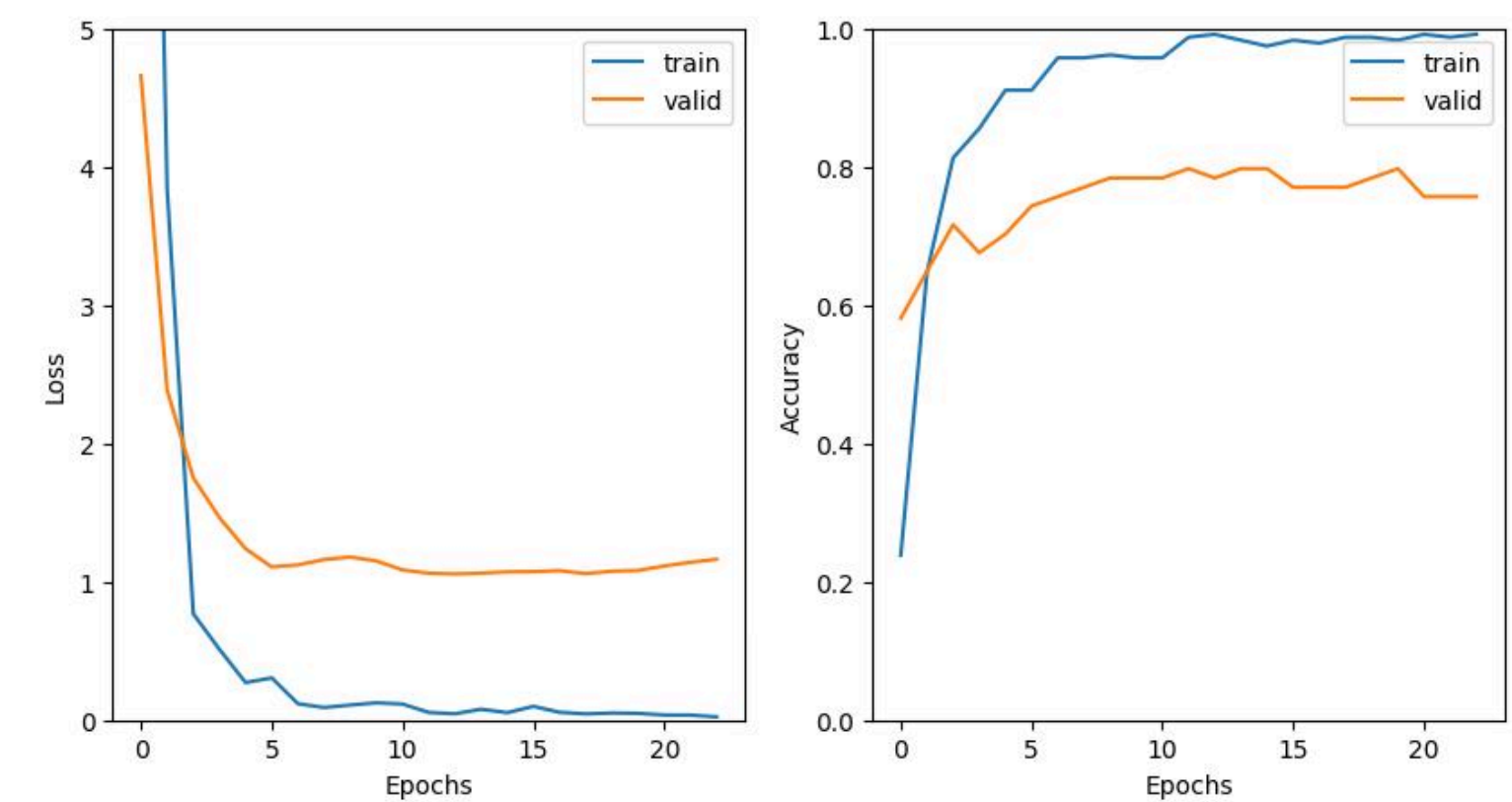
ResNet50

- Accuracy: 70.27 % Loss : 2.2495



EfficientNetB0

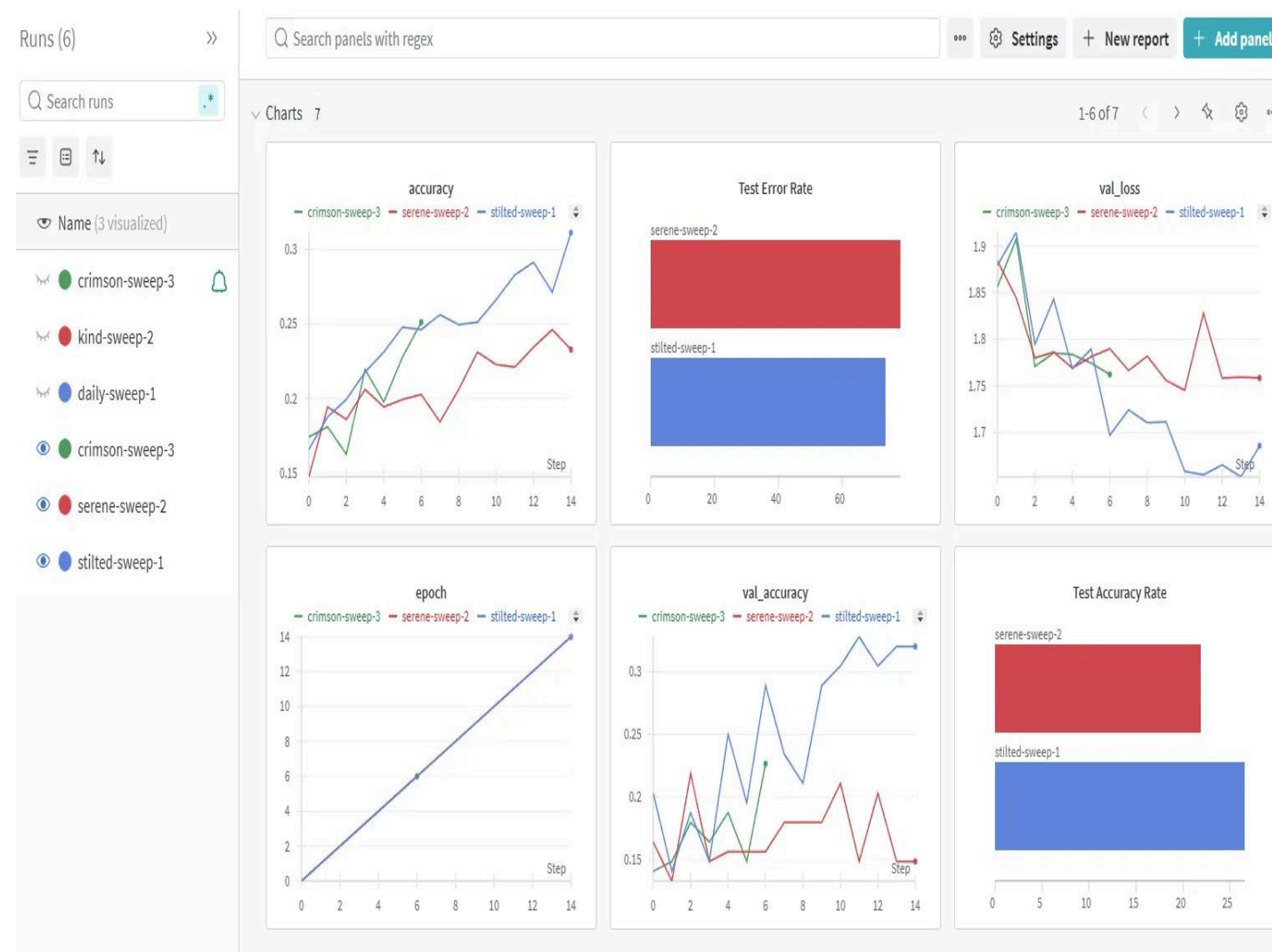
- Accuracy: 75.68 % Loss : 1.0104





04

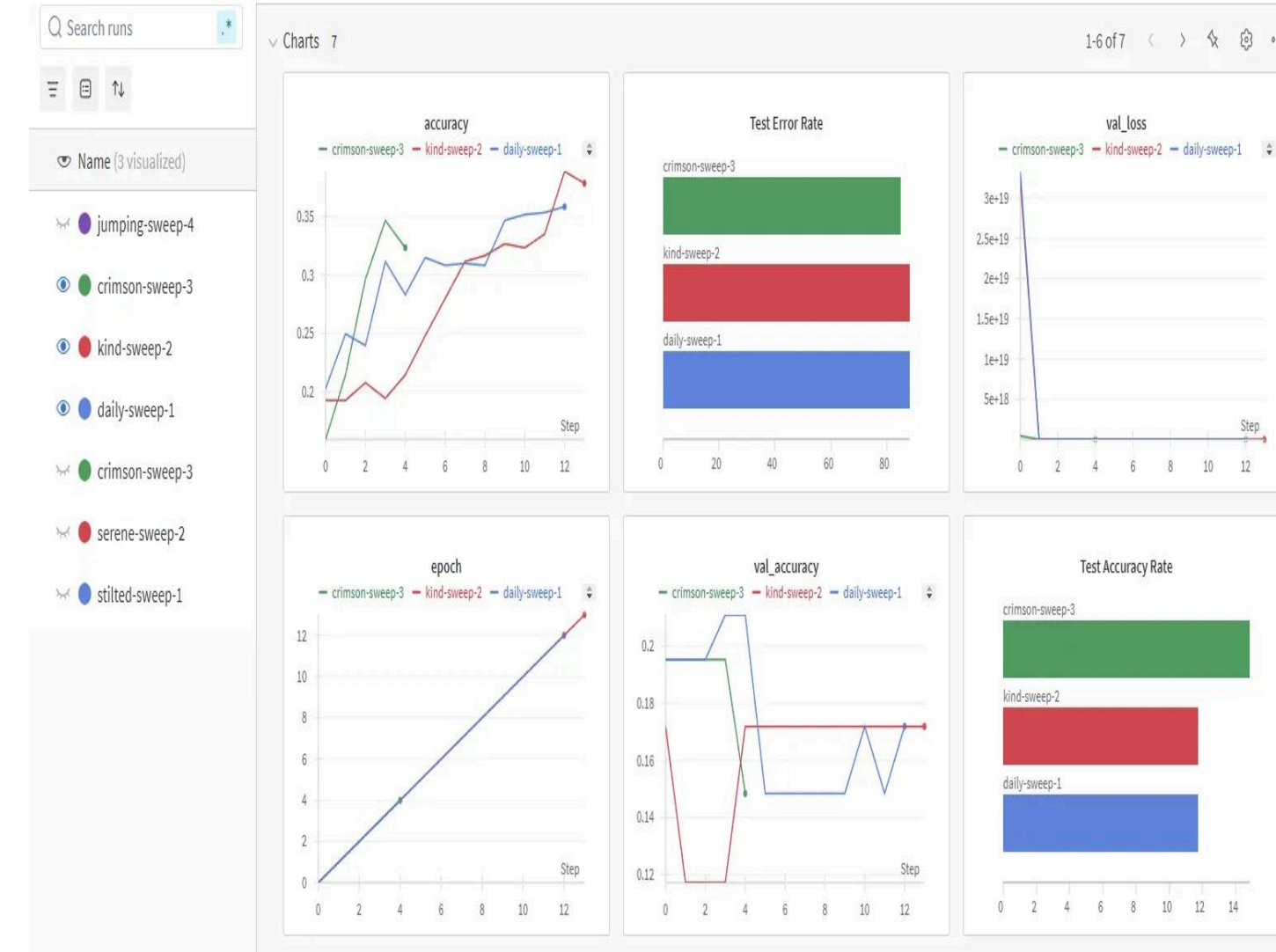
ResNet50 과 W&B



1회차

전처리를 하지 않은 데이터 셋으로 맛보기 테스트를 진행

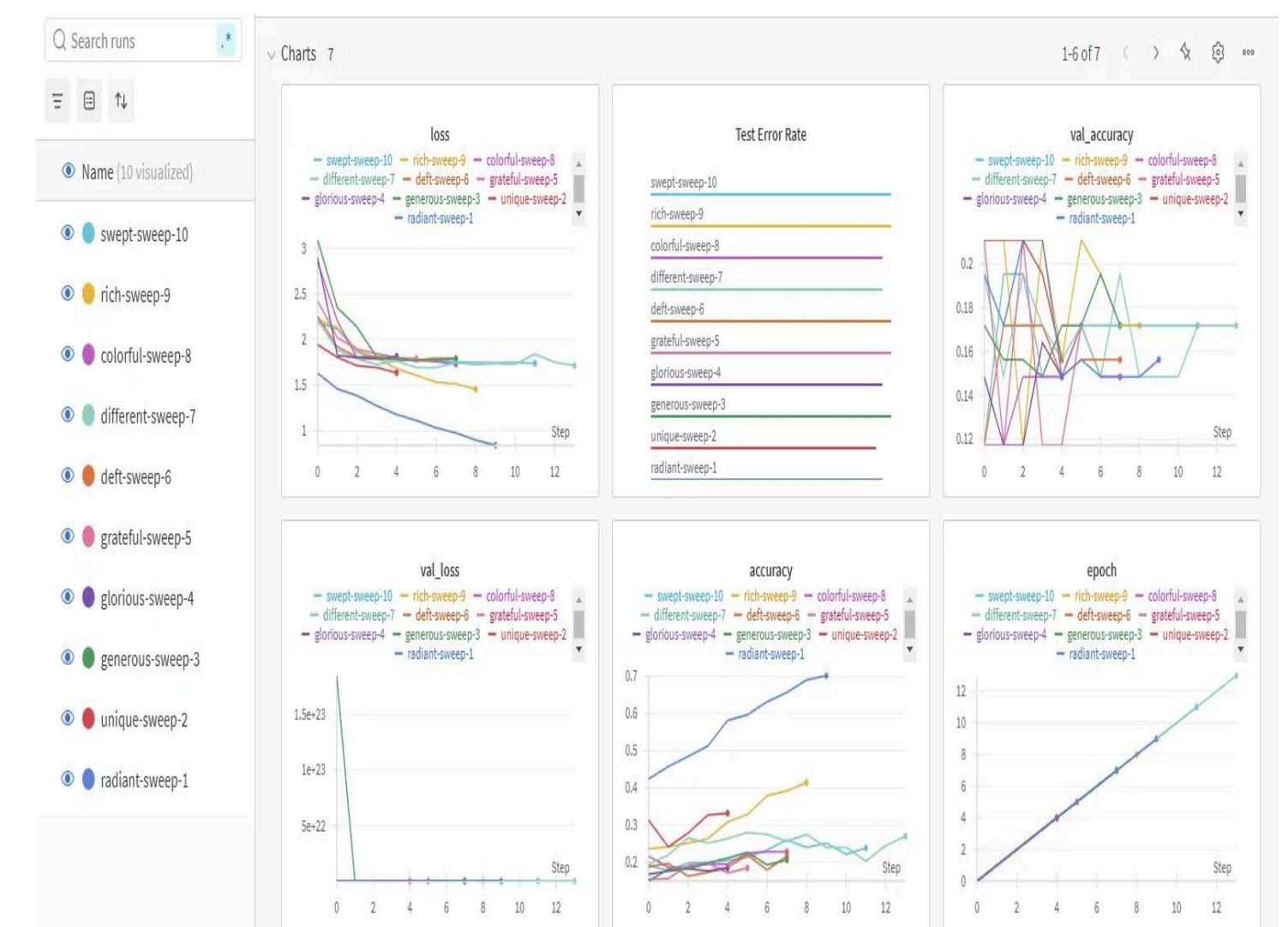
val_acc 가 너무 낮아 코드문제 확인을 위해 중단



2회차

데이터 증강 및 하이퍼 파라미터 수정 후 테스트

```
sweep_config = {  
  "learning_rate": {"min": 0.0001, "max": 0.01},  
  "epoch": {"distribution": "int_uniform", "min": 5, "max": 15},  
  "batch_size": {"values": [16, 32]} }
```



ResNet50v2. 1회차

ResNet50v2 로 변경

에포크 10회 학습

동일한 하이퍼 파라미터 설정

05



Model Training and Transfer Learning



05

Model in Modular Design based on EfficientNet V2L



```
# designing the model
```

```
def eV2L_midsize3(): # this can be adjusted to be hyper-tunable
    inputs = keras.Input(shape=(224, 224, 3))
    x = utils.image_preprocessing_v1()(inputs)
    x = models_base.EV2L((224, 224, 3))(x, training=False)
    x = models_top.midsize3()(x)
    outputs = layers.Dense(6, activation='softmax', name="classifier")(x)

    model = keras.Model(inputs, outputs)

    model.compile(loss='categorical_crossentropy',
                  optimizer=keras.optimizers.RMSprop(learning_rate=1e-4),
                  metrics=['accuracy'])

    return model

eV2L_midsize3().summary()
```

```
def image_preprocessing_v1():
    model = keras.Sequential([
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(0.2),
        layers.RandomTranslation(0.1, 0.1),
        layers.RandomZoom(0.2),

        layers.Normalization(),
    ])

    model.name = "image_preprocessing_v1"

    return model
```

```
import keras

def EV2L(image_shape=(480, 480, 3), trainable=False):
    efficientNetV2L = keras.applications.EfficientNetV2L(
        weights='imagenet',
        include_top=False,
        input_shape=image_shape
    )

    efficientNetV2L.trainable = trainable

    return efficientNetV2L
```

```
def midsize3():
    inputs = keras.Input(shape=(7, 7, 1280))

    x = layers.GlobalAveragePooling2D()(inputs)
    x = layers.Dense(1024, activation='relu')(x)

    residual = x
    x = layers.Dense(1024, activation='relu')(x)
    x = layers.Dense(1024, activation='relu')(x)
    x = layers.Dropout(0.5)(x)
    x = layers.add([x, residual])
    x = layers.Dense(512, activation='relu')(x)
    x = layers.BatchNormalization()(x)

    residual = x
    x = layers.Dense(512, activation='relu')(x)
    x = layers.Dense(512, activation='relu')(x)
    x = layers.Dropout(0.5)(x)
    x = layers.add([x, residual])
    x = layers.Dense(256, activation='relu')(x)
    output = layers.BatchNormalization()(x)

    model = keras.Model(inputs=inputs, outputs=output, name="basic_midsize3")
    return model
```



Trying different Top Models

```
def simple1():
    model = keras.Sequential([
        layers.GlobalAveragePooling2D(),
        layers.Dense(256, activation='relu'),
        layers.Dropout(0.5)
    ])

    model.name = "basic_simple1"

    return model
```

```
def midsize1():
    model = keras.Sequential([
        layers.GlobalAveragePooling2D(),
        layers.Dense(512, activation='relu'),
        layers.Dense(512, activation='relu'),
        layers.Dropout(0.5),
        layers.Dense(256, activation='relu')
    ])

    model.name = "basic_midsize1"

    return model
```

```
def midsize3():
    inputs = keras.Input(shape=(7, 7, 1280))

    x = layers.GlobalAveragePooling2D()(inputs)
    x = layers.Dense(1024, activation='relu')(x)

    residual = x
    x = layers.Dense(1024, activation='relu')(x)
    x = layers.Dense(1024, activation='relu')(x)
    x = layers.Dropout(0.5)(x)
    x = layers.add([x, residual])
    x = layers.Dense(512, activation='relu')(x)
    x = layers.BatchNormalization()(x)

    residual = x
    x = layers.Dense(512, activation='relu')(x)
    x = layers.Dense(512, activation='relu')(x)
    x = layers.Dropout(0.5)(x)
    x = layers.add([x, residual])
    x = layers.Dense(256, activation='relu')(x)
    output = layers.BatchNormalization()(x)

    model = keras.Model(inputs=inputs, outputs=output, name="basic_midsize3")
    return model
```



05

TOP MODEL #1

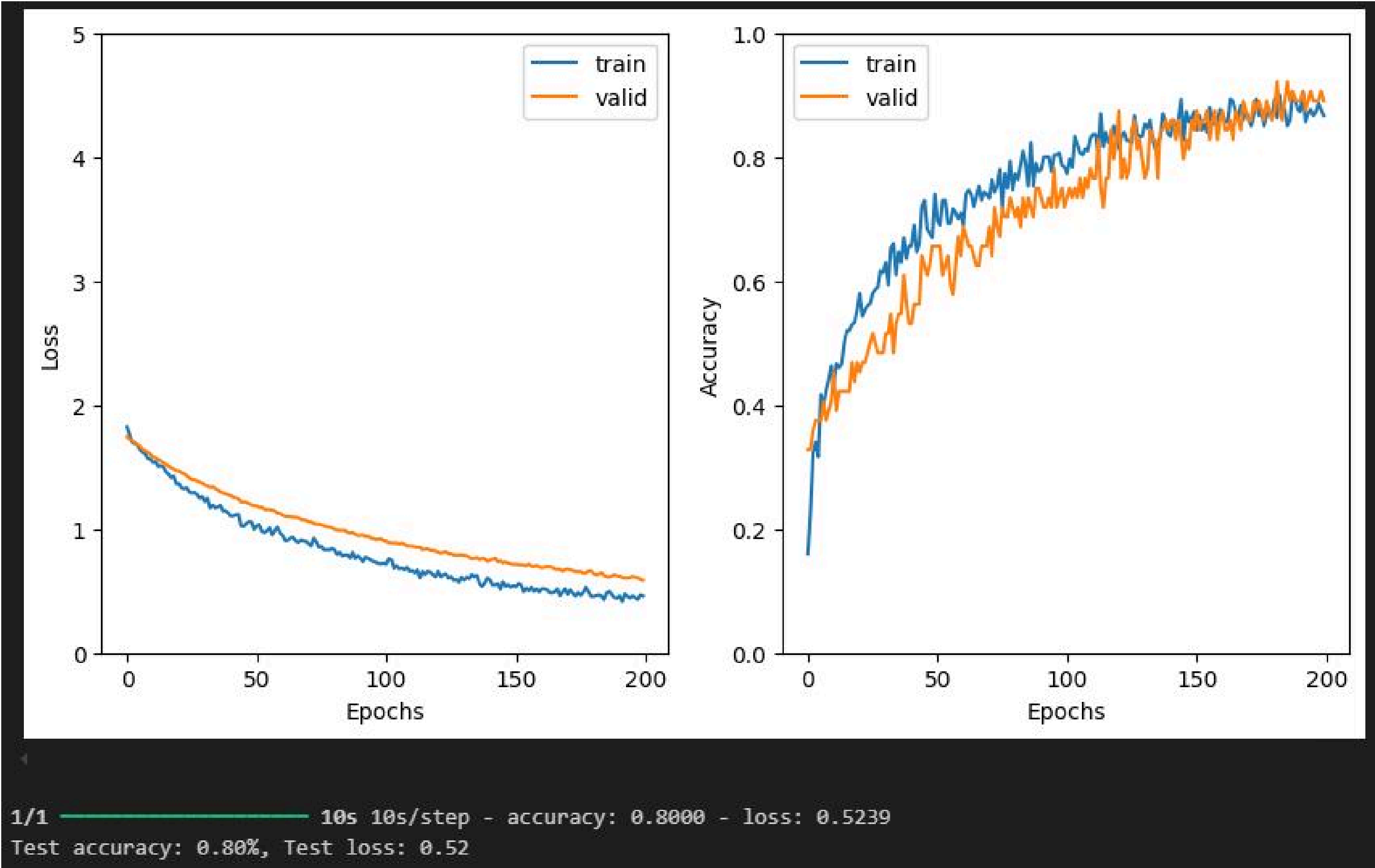
Simple1

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 3)	0
image_preprocessing_v1 (Sequential)	(None, 224, 224, 3)	7
efficientnetv2-l (Functional)	(None, 7, 7, 1280)	117,746,848
basic_simple1 (Sequential)	(None, 256)	327,936
classifier (Dense)	(None, 6)	1,542

```
def simple1():
    model = keras.Sequential([
        layers.GlobalAveragePooling2D(),
        layers.Dense(256, activation='relu'),
        layers.Dropout(0.5)
    ])

    model.name = "basic_simple1"

    return model
```





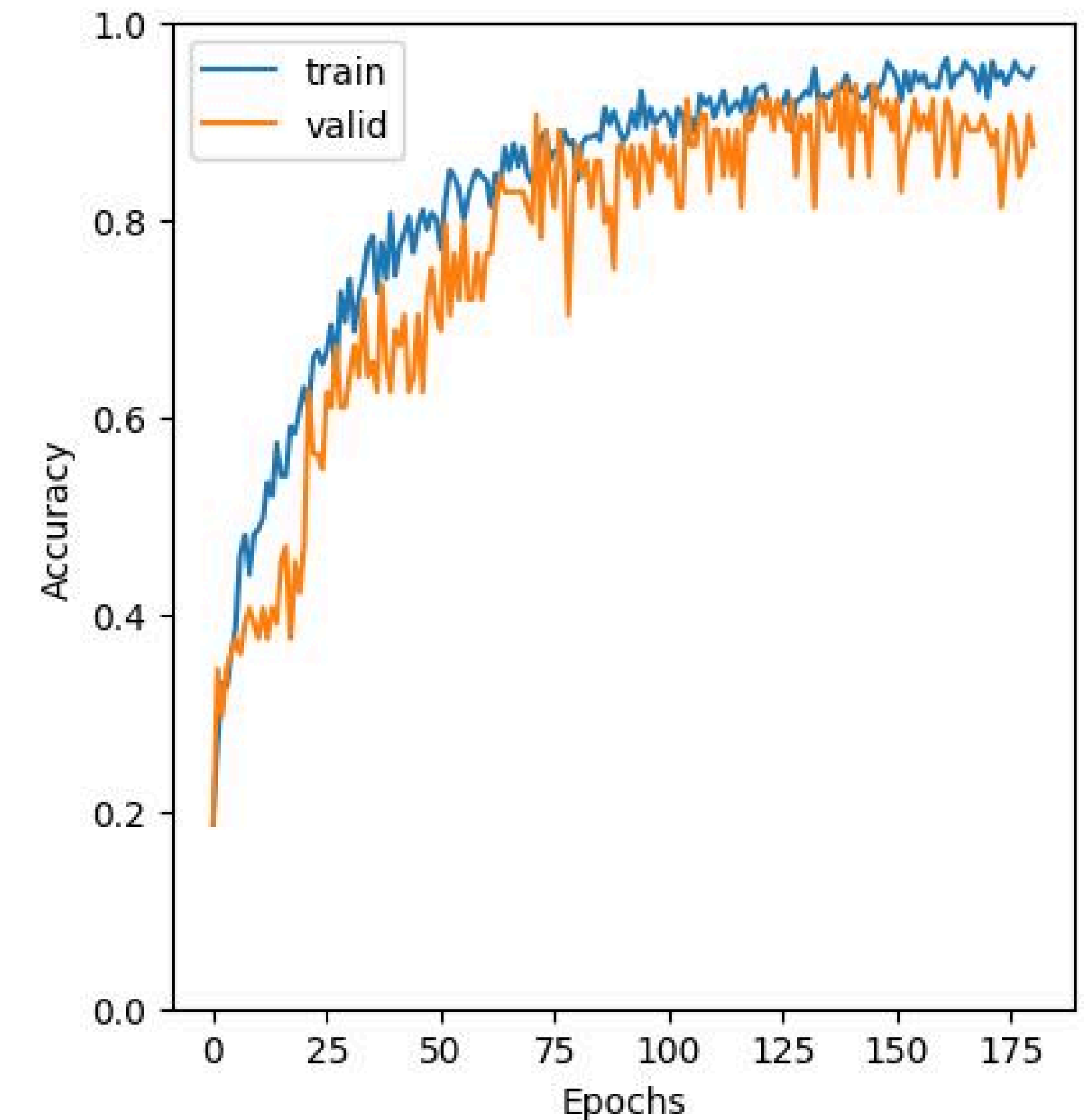
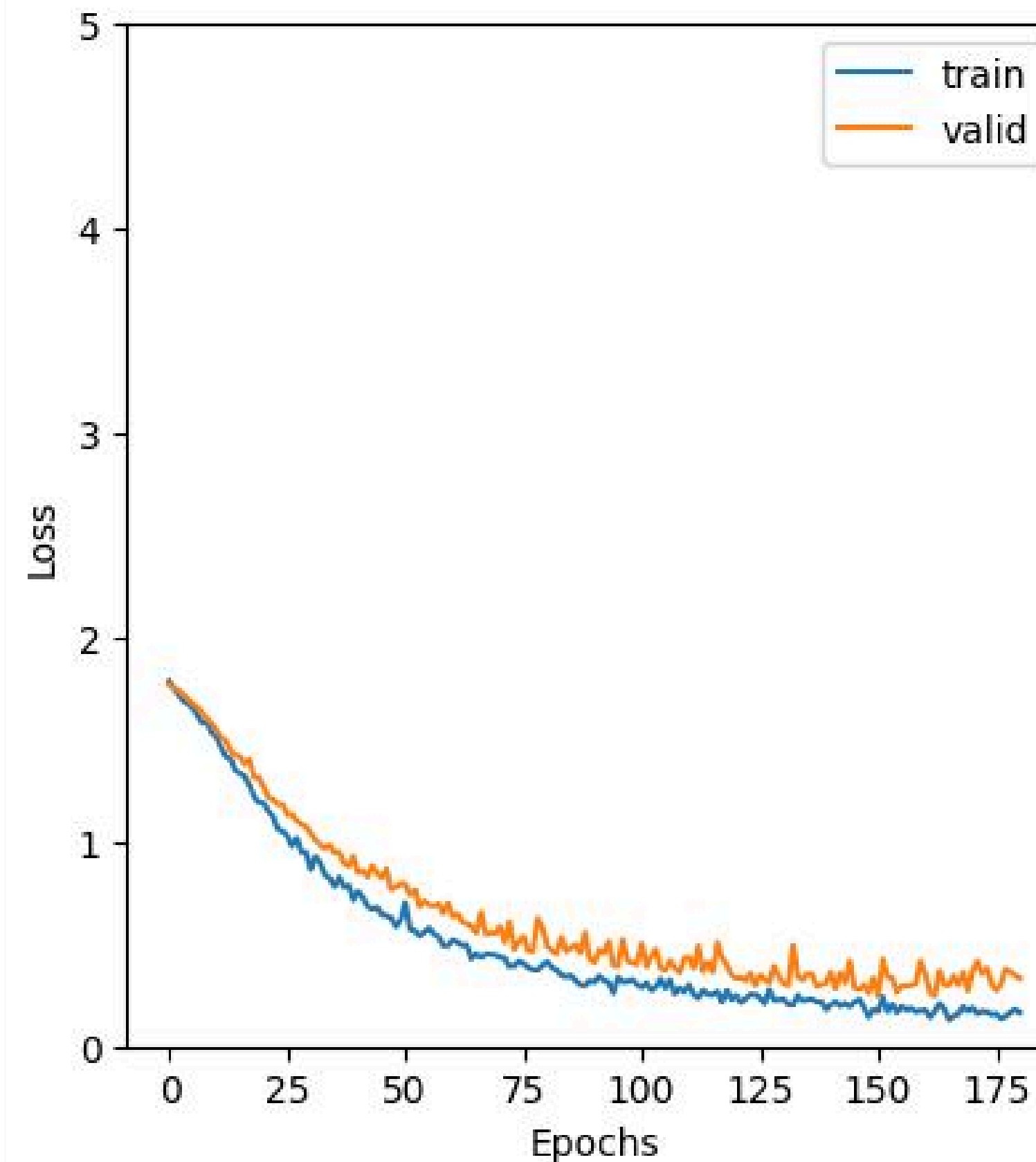
05

TOP MODEL #2

Midsize1

Layer (type)	Output Shape	Param #
input_layer_8 (InputLayer)	(None, 224, 224, 3)	0
image_preprocessing_v1 (Sequential)	(None, 224, 224, 3)	7
efficientnetv2-l (Functional)	(None, 7, 7, 1280)	117,746,848
basic_midsize1 (Sequential)	(None, 256)	1,049,856
classifier (Dense)	(None, 6)	1,542

```
def midsize1():  
    model = keras.Sequential([  
        layers.GlobalAveragePooling2D(),  
        layers.Dense(512, activation='relu'),  
        layers.Dense(512, activation='relu'),  
        layers.Dropout(0.5),  
        layers.Dense(256, activation='relu')  
    ])  
  
    model.name = "basic_midsize1"  
  
    return model
```



1/1 ————— 9s 9s/step - accuracy: 0.8000 - loss: 0.4869
Test accuracy: 0.80%, Test loss: 0.49



05

TOP MODEL #3

Midsize3

Layer (type)	Output Shape	Param #
input_layer_24 (InputLayer)	(None, 224, 224, 3)	0
image_preprocessing_v1 (Sequential)	(None, 224, 224, 3)	7
efficientnetv2-l (Functional)	(None, 7, 7, 1280)	117,746,848
basic_midsize3 (Functional)	(None, 256)	4,594,432
classifier (Dense)	(None, 6)	1,542

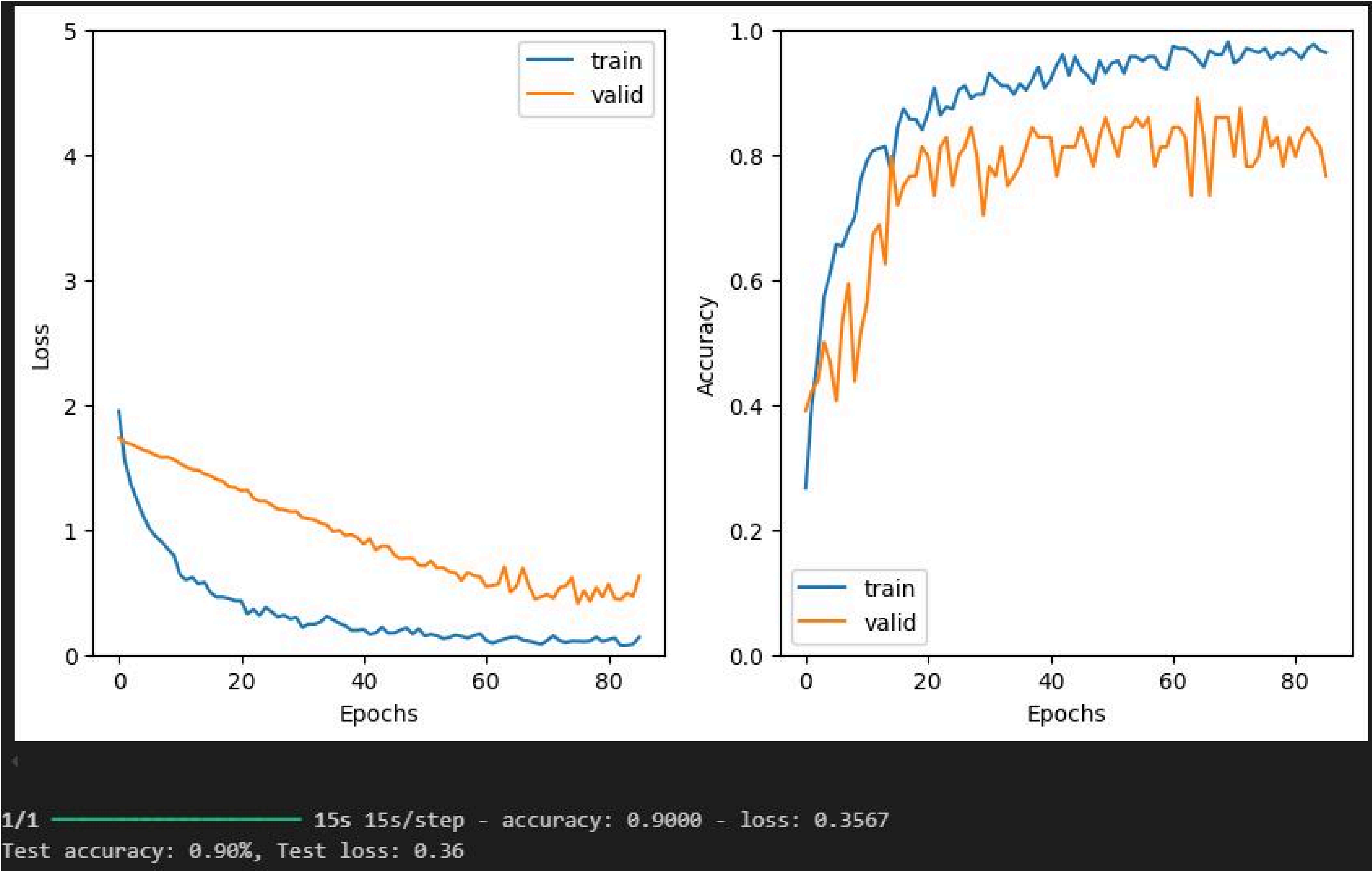
```
def midsize3():
    inputs = keras.Input(shape=(7, 7, 1280))

    x = layers.GlobalAveragePooling2D()(inputs)
    x = layers.Dense(1024, activation='relu')(x)

    residual = x
    x = layers.Dense(1024, activation='relu')(x)
    x = layers.Dense(1024, activation='relu')(x)
    x = layers.Dropout(0.5)(x)
    x = layers.add([x, residual])
    x = layers.Dense(512, activation='relu')(x)
    x = layers.BatchNormalization()(x)

    residual = x
    x = layers.Dense(512, activation='relu')(x)
    x = layers.Dense(512, activation='relu')(x)
    x = layers.Dropout(0.5)(x)
    x = layers.add([x, residual])
    x = layers.Dense(256, activation='relu')(x)
    output = layers.BatchNormalization()(x)

    model = keras.Model(inputs=inputs, outputs=output, name="basic_midsize3")
    return model
```





Conclusion

1. A Good Data is All You Need:

- a. No improvement in architecture and model tuning is better than a good, well-sorted data
- b. Re-labeling 5 mislabeled data alone increased ~10% accuracy

2. The Bigger the Model is, the Faster it Overfitted

- a. With big enough model, every dataset is overfit-able
- b. If you find the dataset is too small to be generalized, make your model bigger

3. Don't be afraid to start over



9:56



Jelly Pop



 사진 찾아보기



Q&A

Thank You!

